Stock Price Prediction with a LSTM

1 Dataset Origin and Explanation

The dataset used is a two year period of stock data of the AMC stock. It starts two years from 04-06-2026 and ends on that day. The dataset has daily data on open, close, high, and low price of the day, as well as the total volume of that day. It is extracted from Yahoo Finance through the yfinance Python library.

1.1 Technical Description

The objective of the LSTM model is to predict the close price of the AMC stock of the immediate next day after the dataset's end, given the data on the last 12 days. The model is trained on two years worth of data, to allow it to generalize on the common stock movements.

The TensorFlow library was also used to create the LSTM model, which consists of a Sequential architecture containing an LSTM layer, a dropout layer and a Dense layer.

1.1.1 Data Preparation

To prepare the data, the 5 features are scaled on a scale from 0 to 1, so that they all have the same weight in the decisions. This is especially important since volume is orders of magnitude higher than any of the other 4 variables.

1.1.2 Model Architecture

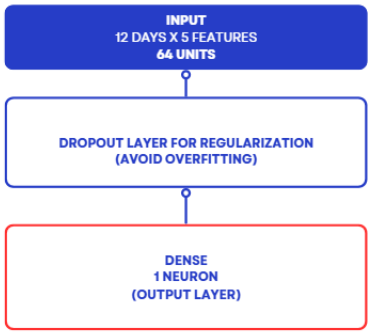
The model takes an input of shape (12, 5) because it looks back 12 days, and has 5 features. A LSTM layer of 64 units was chosen because there needs to be a balance between capacity (more neurons) and overfitting (less neurons), the LSTM layer allows us to learn from day-to-day without forgetting or amplifying signal from a few days back. A dropout of 0.2 randomly disables 20% of neurons during training, acting as regularization to prevent the model from memorizing the training data, which would mean that it has overfitted and significantly harm its performance.

The final Dense(1) layer collapses the LSTM output to a single value. This value is the predicted close price of the AMC stock for the following day.

1.1.3 Configurations and Training

An Adam optimizer is used because it has an adapivelearning rate, and the MSE is the chosen loss function as it penalizes larger prediction errors more heavily. This is desired for our finantial data, as making a large mistake should be avoided, it is more desirable to make more small mistakes than one large mistake. However, this could vary depending on the goal of the model.

The following schematic shows the model layers:



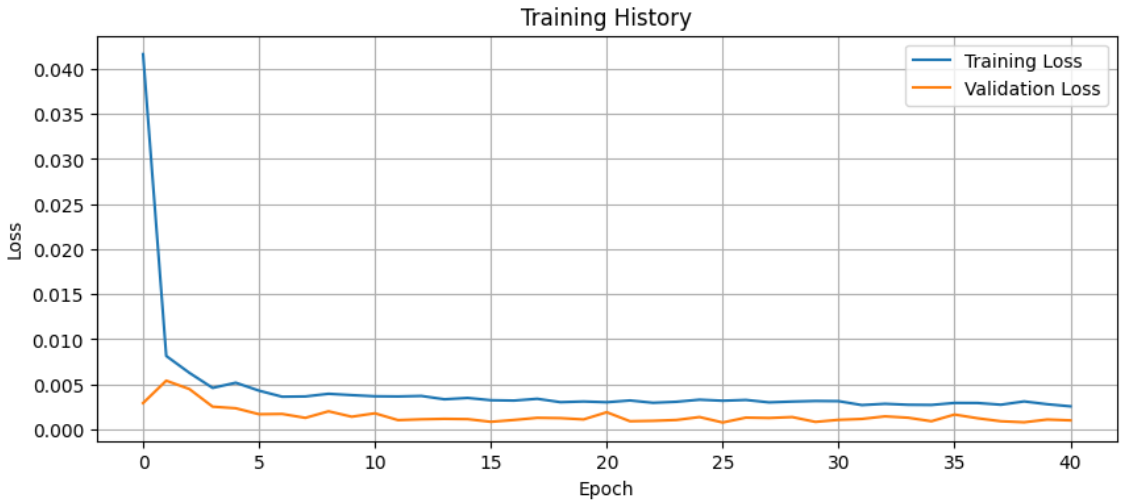
The model is trained on a maximum of 200 epochs with an early stopping feature so the training is stopped once the epochs stop benefitting its performance. The model stops training once 15 epochs provide no benefit. The final number of epochs with the setup is 40.

The training is done in batches of 16 to make training faster, and more stable and quality gradients to update weights.

1.2 Results of the Algorithms and Analysis of the Results

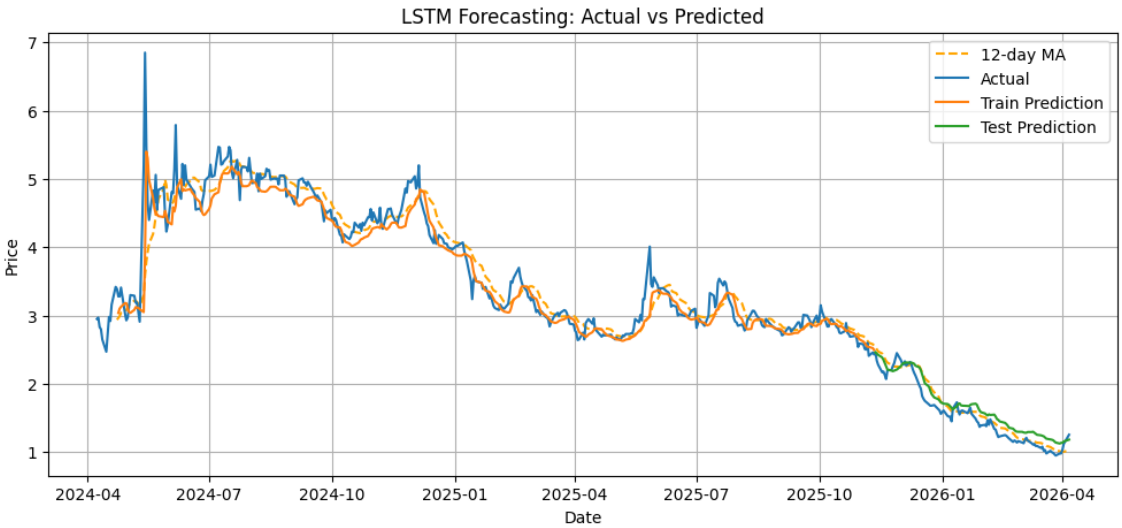
1.2.1 Training Epoch Evolution

The folliowing chart displays the loss over the epochs for the train and validation sets. We can see there varely is any more learning after epoch 10, while the first 5 epochs are the most important ones for the training.



1.2.2 Actual Prices and Prediction for Train and Test Prices

The following chart has the actual price, as well as the 12-day Moving Average and the LSTM's prediction. We can see the LSTM does not work just like the moving average, but it is closer to that than to the actual stock price, which fluctuates much more. This is expected from the LSTM architecture which softens new changes thanks to the long-term memory.



The model predicts a closing price of **$1.24** on the next close, which could well be the case, but of course is unpredictable due to the complexity of the stock market.

1.2.3 Actual Prices and Prediction for Train and Test Prices

The test RMSE is 0.16 and the MAE is 0.15. This similarity between the two metrics indicates common small errors, but not very large ones, which is good for a stock market model, and is expected as well.

1.3 CODE:

```
# Data Collection
import yfinance as yf
ticker = 'AMC'

data = yf.download(ticker, period="2y", interval="1d")

# Evolution of the Stock Price
plt.figure(figsize=(12, 4))
plt.plot(data.index, data["Close"])
plt.title("Yearly evolution of stock price")
plt.xticks(rotation=90)
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.grid(True)
plt.show()

# Processing the data to facilitate learning.
# I used two scalers so later it's easier to do inverse scaling
close_scaler = MinMaxScaler(feature_range=(0, 1))
other_scaler = MinMaxScaler(feature_range=(0, 1))

scaled close = close scaler.fit transform(data[['Close']])
scaled_others = other_scaler.fit_transform(data[['Open', 'Low', 'High', 'Volume']])

scaled data = np.hstack([scaled close, scaled others])

def create_data_and_labels(dataset, look_back=12):
    X, y = [], []
    for i in range(len(dataset) - look_back):
        X.append(dataset[i:i + look_back, :])
        y.append(dataset[i + look_back, 0])         # Only the close is predicted)
    return np.array(X), np.array(y)

X, y = create_data_and_labels(scaled_data, look_back)

# 5 features
train_size = int(len(X) * 0.8)
```

```
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# LSTM expects input in the form: (samples, time_steps, features)
X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 5))
X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 5))


tf.random.set_seed(42)
np.random.seed(42)

model = Sequential([
    LSTM(64, input_shape=(look_back, 5)),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.summary()


early_stop = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=16,
    validation_data=(X_test, y_test),
    callbacks=[early_stop],
    verbose=1
)

# Loss over epochs plot
plt.figure(figsize=(10, 4))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training History")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

# Training and testing predictions for plot
train_pred = model.predict(X_train, verbose=0)
```

```
test_pred = model.predict(X_test, verbose=0)

# inverse transform predictions and actual values
train_pred_inv = close_scaler.inverse_transform(train_pred)
test_pred_inv = close_scaler.inverse_transform(test_pred)

y_train_inv = close_scaler.inverse_transform(y_train.reshape(-1, 1))
y_test_inv = close_scaler.inverse_transform(y_test.reshape(-1, 1))
print("train_pred range:", train_pred.min(), train_pred.max())
print("test_pred range:", test_pred.min(), test_pred.max())
print("y_train range:", y_train.min(), y_train.max())  # should be 0-1
print("model output shape:", train_pred.shape)  # should be (-1, 1)

# Plot Predictions for trianing and test sets
train_plot = np.full((len(scaled_data), 1), np.nan)
test_plot = np.full((len(scaled_data), 1), np.nan)

# place predictions in the correct time positions
train_plot[look_back:look_back + len(train_pred_inv)] = train_pred_inv
test_plot[look_back    +    len(train_pred_inv):look_back    +    len(train_pred_inv)    +
len(test_pred_inv)] = test_pred_inv

actual_values = close_scaler.inverse_transform(scaled_data[:, 0].reshape(-1,1))

plt.figure(figsize=(12, 5))
ma_12 = data['Close'].rolling(window=12).mean().values
plt.plot(data.index, ma_12, label="12-day MA", linestyle="--", color="orange")
plt.plot(data.index, actual_values, label="Actual")
plt.plot(data.index, train_plot, label="Train Prediction")
plt.plot(data.index, test_plot, label="Test Prediction")
plt.title("LSTM Forecasting: Actual vs Predicted")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid(True)
plt.show()




# Metrics

train_rmse = np.sqrt(mean_squared_error(y_train_inv, train_pred_inv))
test_rmse = np.sqrt(mean_squared_error(y_test_inv, test_pred_inv))

train_mae = mean_absolute_error(y_train_inv, train_pred_inv)
test_mae = mean_absolute_error(y_test_inv, test_pred_inv)

# Prediction of following day close price
```

```
last_sequence = scaled_data[-look_back:]  # shape (12, 5)
last_sequence = last_sequence.reshape(1, look_back, 5)  # shape (1, 12, 5)

next_day_scaled = model.predict(last_sequence, verbose=0)

next_day_price = close_scaler.inverse_transform(next_day_scaled)
print(f"Predicted next day close for {ticker}: ${next_day_price[0][0]:.2f}")
```